한국산업은행(KDB) IT 스터디 상세 해설 및 서술 평가기준

DS · 알고리즘 집중 세트 (총 48점 만점) | 논술형 1번: Greedy vs DP (40점) · 약술형 2번: 최소 힙(4점) · 약술형 3번: MST(4점)

1. [논술형 1번] Greedy, DP — 0-1 배낭 채우기 문제 상세 해설 및 채점기준 (40점)

출제 의도 및 2026 하반기 복기본 대비 변형·신규 포인트

- **기출 연결성 (2026 하반기 1번 40점 완벽 대응):** 2026 하반기 산업은행 복기본 1번(`Greedy vs DP 원단 자르기`)의 소문항 4개 구조(`1-1 개념 및 해결방식 비교 → 1-2 탐욕 알고리즘 적용 및 한계 → 1-3 DP 점화식 의미 및 테이블 완성 → 1-4 단순 재귀 vs DP 동작·복잡도`)를 100% 동일하게 유지했습니다.
- **핵심 변형 포인트:** 1차원 배열을 채우던 원단 자르기에서 한 단계 발전시켜, 실제 시험에서 변별력을 가르는 **2차원 DP 테이블(0-1 Knapsack, 허용 무게 5kg)**과 **1차원 공간 최적화 시 역순 순회 원리**까지 점검할 수 있도록 설계했습니다.
- **수치 설계의 묘미:** 물품 `A(1kg, 6만 원, 단위 6.0)`, `B(2kg, 10만 원, 단위 5.0)`, `C(3kg, 12만 원, 단위 4.0)`, `D(3kg, 9만 원, 단위 3.0)`로 설정하여, 단위 가치 순으로 담는 그리디(`{A, B} = 3kg, 16만 원`, 남은 2kg 유휴 공간 발생)와 DP 최적해(`{B, C} = 5kg, 22만 원`)의 차이가 직관적으로 드러나도록 구성했습니다.

[1-1] 모범 답안 및 서술 평가기준 (10점)

- **문제 요약:** 분할 가능 배낭 문제(Fractional Knapsack)와 0-1 배낭 문제(0-1 Knapsack)의 개념·차이점 설명 및 적합한 알고리즘(탐욕 vs DP)과 그 이유를 최적 부분 구조·탐욕적 선택 속성 관점에서 서술.
- **상세 모범 답안:**
  1. **개념 및 차이점: 분할 가능 배낭 문제**는 물품을 쪼개어 무게의 일부(분수 형태)만 배낭에 담을 수 있는 문제이며, **0-1 배낭 문제**는 물품을 쪼갤 수 없어 각 물품을 통째로 담거나(`1`) 아예 담지 않는(`0`) 이진 선택만 가능한 문제이다.
  2. **적합한 알고리즘 및 원리 비교:**
     - 두 문제 모두 큰 문제의 최적해가 부분 문제의 최적해들로 구성되는 **최적 부분 구조(Optimal Substructure)**를 만족한다.
     - **분할 가능 배낭 문제 → 탐욕 알고리즘(Greedy Algorithm):** 단위 무게당 가치(`$V_i$ / $W_i$`)가 가장 높은 물품부터 우선적으로 담고, 남은 용량보다 물품이 무거우면 남은 용량만큼만 쪼개어 배낭을 100% 꽉 채울 수 있다. 즉, 매 순간의 최선 선택이 전체 최적해로 이어지는 **탐욕적 선택 속성(Greedy Choice Property)**이 성립하므로 탐욕 알고리즘으로 최적해를 구할 수 있다.
     - **0-1 배낭 문제 → 동적 계획법(Dynamic Programming):** 물품을 쪼갤 수 없기 때문에 단위 무게당 가치가 가장 높은 물품을 먼저 담으면 배낭에 채우지 못하는 **자투리 빈 공간(유휴 용량)**이 발생할 수 있어 **탐욕적 선택 속성이 성립하지 않는다.** 따라서 각 물품을 포함하는 경우와 포함하지 않는 경우의 부분 문제 해를 테이블에 기록하며 비교하는 **동적 계획법(DP)**으로 해결해야 한다.

평가 항목 (채점 키워드)	배점	교차채점 상세 기준 (부분점수 가이드)
분할 가능 vs 0-1 배낭 차이	**2점**	물품 분할(쪼개기) 가능 여부(연속적 일부 담기 vs 0 또는 1 선택)를 명확히 대비하여 설명 시 2점
적합한 알고리즘 매칭	**3점**	분할 가능 배낭 = 탐욕 알고리즘(1.5점), 0-1 배낭 = 동적 계획법(1.5점)

평가 항목 (채점 키워드)	배점	교차채점 상세 기준 (부분점수 가이드)
최적 부분 구조 & 탐욕적 선택 속성 관점 서술	**5점**	• **최적 부분 구조**(공통)와 **탐욕적 선택 속성**(분할 가능은 성립, 0-1은 유휴 용량 발생으로 불성립)을 모두 명시하여 서술 시 **5점** • 용어가 일부 생략되었더라도 "0-1 배낭은 물건을 쪼갤 수 없어 단위 가치가 높은 것을 먼저 넣으면 남은 빈 공간 때문에 전체 가치가 작아질 수 있다"고 정확히 설명 시 **3.5점**

[1-2] 모범 답안 및 서술 평가기준 (10점)

- **문제 요약:** 단위 무게당 가치가 높은 순서대로 담는 탐욕 알고리즘 적용 시 선택 물품 조합, 총 무게, 총 가치를 단계별로 구하고 최적해가 아닌 이유 설명.
- **상세 모범 답안:**
  1. **단위 무게당 가치 정렬:** `A(6.0) > B(5.0) > C(4.0) > D(3.0)` 순서로 검사한다(초기 배낭 잔여 용량 = **5kg**).
  2. **단계별 선택 추적(Trace):**
     - **Step 1 (물품 A 검사, 1kg, 6만 원):** 잔여 용량(5kg) 이하이므로 **선택(O)** → 누적 무게 **1kg**, 누적 가치 **6만 원** (잔여 용량 **4kg**).
     - **Step 2 (물품 B 검사, 2kg, 10만 원):** 잔여 용량(4kg) 이하이므로 **선택(O)** → 누적 무게 **3kg**, 누적 가치 **16만 원** (잔여 용량 **2kg**).
     - **Step 3 (물품 C 검사, 3kg, 12만 원):** 잔여 용량(2kg)보다 무거우므로 **선택 불가(X, 건너뜀)**.
     - **Step 4 (물품 D 검사, 3kg, 9만 원):** 잔여 용량(2kg)보다 무거우므로 **선택 불가(X, 건너뜀)**.
  3. **탐욕 알고리즘 최종 결과:** 선택 물품 조합 `{A, B}`, 총 무게 **3kg**, 총 판매 가치 **16만 원**.
  4. **최적해가 되지 못하는 이유:** 단위 무게당 가치가 가장 높은 `A(1kg, 6만 원)`를 탐욕적으로 먼저 담은 결과, 배낭에 **2kg의 유휴 공간(빈 공간)**이 남게 되어 무게가 **3kg**인 `C(12만 원)`를 담을 수 없게 되었다. 실제로는 **A**를 포기하고 `{B(2kg, 10만 원), C(3kg, 12만 원)}`를 담으면 **5kg** 용량을 100% 활용하여 **22만 원**(탐욕 결과보다 6만 원 더 큰 최적해)을 얻을 수 있다.

평가 항목 (채점 키워드)	배점	교차채점 상세 기준 (부분점수 가이드)
단계별 탐욕 선택 과정 (Step 1~4)	**3점**	단위 가치 순서(A→B→C→D)대로 검사하여 A, B 선택 후 잔여 용량이 2kg이라 C, D를 건너뛰는 과정 명시
탐욕 결과 (조합·무게·총 가치)	**4점**	선택 물품 `{A, B}`(1.5점), 총 무게 **3kg**(0.5점), 총 가치 **16만 원**(2점)
최적해가 아닌 이유 및 실제 최적해 대조	**3점**	**A** 선택으로 인해 **2kg**의 빈 공간이 낭비된다는 점(1.5점) + 실제 최적해인 `{B, C} = 22만 원`을 제시하여 비교(1.5점)

[1-3] 모범 답안 및 서술 평가기준 (12점)

- **문제 요약:** 점화식의 `DP[i][w]`, `DP[i-1][w]`, `V[i] + DP[i-1][w - W[i]]` 의미 설명 및 `DP[1..4][0..5]` 테이블 완성, 최대 가치와 물품 조합 도출.
- **상세 모범 답안:**
  1. **점화식 각 항의 의미:**
     - `DP[i][w]`: 첫 번째 물품부터 `i`번째 물품까지만 고려하고 배낭의 허용 무게 한도가 `w`일 때 얻을 수 있는 **최대 가치**.
     - `DP[i-1][w]`: 현재 `i`번째 물품을 **배낭에 넣지 않는 경우(미포함)**의 최대 가치(`i-1`번째 물품까지로 무게 `w`를 채운 최적해를 그대로 가져옴).
     - `V[i] + DP[i-1][w - W[i]]`: 현재 `i`번째 물품을 **배낭에 넣는 경우(포함)**의 최대 가치(현재 물품의 무게 `W[i]` 만큼을 뺀 남은 용량 `w - W[i]`에 대해 `i-1`번째 물품까지로 구한 최적 가치에 현재 물품의 가치 `V[i]`를 더한 값).

2. `DP[i][w]` 전체 테이블 계산 결과:

구분 (물품 i \ 허용 무게 w)	w = 0	w = 1	w = 2	w = 3	w = 4	w = 5	행별 핵심 갱신 계산 근거
i = 0 (초기값)	0	0	0	0	0	0	물품이 없을 때 모든 가치를 0으로 초기화
i = 1 (A: 1kg, 6만 원)	0	6	6	6	6	6	w ≥ 1 구간에서 물품 A(6만 원) 적재 가능
i = 2 (B: 2kg, 10만 원)	0	6	10	16	16	16	w=2: max(6, 10+0)=10 / w≥3: max(6, 10+6)=16 (A+B)
i = 3 (C: 3kg, 12만 원)	0	6	10	16	18	**22**	w=3: max(16, 12+0)=16 / w=4: max(16, 12+6)=18 (A+C) / **w=5: max(16, 12+10)=22 (B+C)**
i = 4 (D: 3kg, 9만 원)	0	6	10	16	18	**22**	w=5: max(DP[3][5]=22, 9+DP[3][2]=19) = **22 유지**

- **최대 가치 및 선택 물품 조합:**
  - **최대 판매 가치 (`DP[4][5]`): `22만 원`**
  - **달성 방법(물품 선택 조합): 물품 B(2kg, 10만 원) + 물품 C(3kg, 12만 원)** (역추적: `DP[4][5]=22 == DP[3][5]` 이므로 D 미포함 → `DP[3][5]=22 != DP[2][5]=16`이므로 **C 포함** 및 남은 무게 `5-3=2`로 이동 → `DP[2][2]=10 != DP[1][2]=6`이므로 **B 포함** 및 남은 무게 `2-2=0` 종료).

평가 항목 (채점 키워드)	배점	교차채점 상세 기준 (부분점수 가이드)
점화식 3개 항 의미 설명	**3점**	`DP[i][w]` 정의(1점) + `DP[i-1][w]`(i번째 미포함, 1점) + `V[i]+DP[i-1][w-W[i]]`(i번째 포함, 1점)
DP[1..4][0..5] 테이블 작성	**6점**	4개 행(i=1~4) 각 1.5점씩 총 6점 (특히 `DP[3][4]=18`, `DP[3][5]=22`, `DP[4][5]=22` 정확성 중점 확인)
최대 가치 및 물품 조합 제시	**3점**	최대 가치 `22만 원`(1.5점) + 선택 물품 조합 `{B, C}`(1.5점)

[1-4] 모범 답안 및 서술 평가기준 (8점)

- **문제 요약:** 단순 재귀와 DP의 동작 방식 비교(중복 부분 문제 관점), 각각의 시간·공간복잡도 서술, 1차원 배열 최적화 시 역순 순회 이유 설명.
- **상세 모범 답안:**
  1. **동작 방식 비교:**
     - **단순 재귀(완전 탐색):** 각 물품마다 '넣는다 / 넣지 않는다'의 2가지 선택지를 재귀 함수로 분기하여 모든 부분집합(2^N개)을 탐색한다. 이 과정에서 동일한 `(i, w)` 상태인 **중복 부분 문제(Overlapping Subproblems)를 반복해서 다시 계산**하므로 지수 시간의 비효율이 발생한다.
     - **동적 계획법(DP):** 부분 문제의 해를 테이블에 한 번만 계산해 저장(Tabulation / Memoization)하고, 이후 동일한 부분 문제가 등장하면 다시 계산하지 않고 저장된 값을 `O(1)`에 참조하여 중복 연산을 완전히 제거한다.
  2. **시간복잡도 및 공간복잡도 비교:**
     - **단순 재귀 알고리즘:** 시간복잡도 `O(2^N)` (N개 물품 각각 2갈래 분기), 공간복잡도 `O(N)` (재귀 호출 스택의 최대 깊이가 트리의 높이인 N에 비례).
     - **2차원 테이블 DP:** 시간복잡도 `O(N · K)` (크기가 N × K인 테이블의 각 칸을 상수 시간에 계산), 공간복잡도 `O(N · K)` (`(N+1) × (K+1)` 크기의 2차원 배열 메모리 사용).

3. **1차원 배열(`DP[w]`) 공간 최적화 및 역순($K \rightarrow W_i$) 순회 이유:**
   - `i`번째 행을 계산할 때 직전 행(`i-1`)의 값만 필요하므로 크기가 `K+1`인 1차원 배열 하나만 사용하여 **공간복잡도를 `O(K)`로 줄일 수 있다.**
   - 이때 내부 반복문에서 무게 `w`를 **정방향($W_i \rightarrow K$)으로 순회하면**, 현재 `i`번째 물품을 이미 넣어서 갱신된 `DP[w - W_i]` 값을 같은 `i`번째 루프의 뒤쪽(더 큰 `w`)에서 또 참조하게 되어 **1개뿐인 물품을 여러 번 중복해서 담는 오류(무한 배낭 문제처럼 동작)**가 발생한다. 따라서 아직 갱신되지 않은 `i-1` 시점의 값을 참조하기 위해 반드시 **역순(`w = K`부터 W_i까지 감소)으로 순회**해야 한다.

평가 항목 (채점 키워드)	배점	교차채점 상세 기준 (부분점수 가이드)
단순 재귀 vs DP 동작 방식 비교	**2점**	**중복 부분 문제(Overlapping Subproblems)**의 반복 계산 여부 및 테이블 저장·재사용 차이 서술 시 2점
단순 재귀 시간·공간복잡도	**2점**	시간복잡도 $O(2^N)$(1점) + 공간복잡도(스택 깊이) $O(N)$(1점)
2차원 DP 시간·공간복잡도	**2점**	시간복잡도 $O(N \cdot K)$(1점) + 공간복잡도 $O(N \cdot K)$(1점)
1차원 배열 최적화 및 역순 순회 이유	**2점**	공간복잡도 $O(K)$ 절감 명시(1점) + 정방향 순회 시 **동일 물품이 중복 합산되는 것을 방지**하기 위해 역순 순회한다는 원리 서술(1점)

2. [약술형 2번 - 자료구조(DS)] 최소 힙(Min-Heap) 삽입·삭제 및 우선순위 큐 (4점)

출제 의도 및 변형 포인트

- **출제 이유:** 다익스트라, 프림(Prim), 허프만 코딩, 운영체제 우선순위 스케줄링의 기반 자료구조인 **힙(Heap)의 Up-heap(삽입)과 Down-heap(삭제) 과정**을 손으로 직접 추적(Trace)하고 배열 기반 우선순위 큐와 복잡도를 비교하는 4점짜리 핵심 문항입니다.
- **채점 주의사항:** `deleteMin()` 수행 시 루트 자리로 올라온 마지막 노드(`30`)를 두 자식 노드(`15`와 `20`) 중 **더 작은 자식인 `15`와 교환**했는지 확인하는 것이 핵심 채점 포인트입니다.

모범 답안 및 서술 평가기준 (4점)

- **[2-1] 단계별 힙 상태 추적 (2점):**
  1. **키 `30, 20, 10, 40, 15` 순차 삽입 과정 (Up-heap):**
     - `30` 삽입: `[30]`
     - `20` 삽입: 맨 끝에 추가 `[30, 20]` → 부모 `30 > 20`이므로 교환 ⇒ `[20, 30]`
     - `10` 삽입: 맨 끝에 추가 `[20, 30, 10]` → 부모 `20 > 10`이므로 교환 ⇒ `[10, 30, 20]`
     - `40` 삽입: `30`의 왼쪽 자식(index 4)에 추가 → 부모 `30 < 40`이므로 유지 ⇒ `[10, 30, 20, 40]`
     - `15` 삽입: `30`의 오른쪽 자식(index 5)에 추가 `[10, 30, 20, 40, 15]` → 부모 `30 > 15`이므로 교환, 이후 루트 `10 < 15`이므로 종료 ⇒ **최종 삽입 직후 배열:** `[10, 15, 20, 40, 30]`
  2. **`deleteMin()` 1회 수행 과정 (Down-heap):**
     - 루트 최솟값 `10`을 제거하고, 힙의 맨 마지막 원소인 `30`(index 5)을 루트(index 1)로 이동 ⇒ `[30, 15, 20, 40]`
     - 루트 `30`의 왼쪽 자식 `15`(index 2)와 오른쪽 자식 `20`(index 3) 중 **더 작은 값인 `15`와 교환** ⇒ `[15, 30, 20, 40]`
     - 이동한 `30`(index 2)과 그 왼쪽 자식 `40`(index 4)을 비교하면 `30 < 40`으로 최소 힙 속성을 만족하므로 종료 ⇒ **`deleteMin()` 직후 배열:** `[15, 30, 20, 40]`

- **[2-2] 시간복잡도 및 비정렬 배열과의 비교 (2점):**
  - **최소 힙(Min-Heap):** 원소 N개인 완전 이진 트리의 높이가 `O(log N)`이므로, 트리의 높이만큼 비교·교환하는 **삽입(Insertion) 연산은** `O(log N)`, **최솟값 삭제(Deletion) 연산도** `O(log N)`이다.
  - **정렬되지 않은 일반 배열(Unsorted Array):** 배열의 맨 끝 인덱스에 즉시 원소를 추가하면 되므로 **삽입 연산은** `O(1)`로 힙보다 빠르지만, 최솟값을 삭제하려면 배열 전체 원소 N개를 모두 순회하며 최솟값의 위치를 찾아야 하므로 **최솟값 삭제 연산에** `O(N)`이 소요되어 삭제가 빈번한 우선순위 큐에서는 최소 힙이 훨씬 효율적이다.

소문항	평가 항목 (채점 키워드)	배점	교차채점 상세 기준 (부분점수 가이드)
2-1	**5개 키 삽입 직후 상태**	**1점**	1차원 배열 `[10, 15, 20, 40, 30]` 또는 동일한 구조의 이진 트리 그림 작성 시 1점
2-1	**deleteMin() 1회 수행 직후 상태**	**1점**	1차원 배열 `[15, 30, 20, 40]` 또는 동일한 구조의 이진 트리 그림 작성 시 1점
2-2	**최소 힙 삽입·삭제 시간복잡도**	**1점**	삽입 `O(log N)`(0.5점) + 최솟값 삭제 `O(log N)`(0.5점)
2-2	**비정렬 배열 우선순위 큐와 비교**	**1점**	비정렬 배열의 삽입 `O(1)`(0.5점) + 최솟값 삭제 `O(N)`(0.5점) 비교 서술

3. [약술형 3번 - 알고리즘(AL)] 최소 신장 트리(MST): Kruskal vs Prim (4점)

출제 의도 및 변형 포인트

- **출제 이유:** 스터디 대화록에서 신승혜 님이 정리한 산은 필수 대비 주제인 **`MST (프림 ↔ 크루스칼) + Trace + 시간복잡도`**를 4점 약술형으로 제작했습니다.
- **핵심 체크 포인트:** 간선 가중치 순으로 선택할 때 `(A, C) = 3`과 `(B, D) = 5`가 기존 선택 간선들과 **사이클(Cycle)을 형성하므로 기각(Skip)**된다는 점을 정확히 명시하는지 확인합니다.

모범 답안 및 서술 평가기준 (4점)

- **[3-1] 크루스칼(Kruskal) 알고리즘 단계별 Trace 및 MST 가중치 합 (2점):**
  - 정점 수가 `|V| = 5`개이므로 사이클을 만들지 않는 간선 `|V| - 1 = 4`개가 선택될 때까지 가중치 오름차순으로 검사한다.

검사 순서	검사 간선 (가중치)	사이클 발생 여부	채택 / 기각	선택 후 연결된 정점 집합 상태 및 누적 가중치
1	`(A, B) = 1`	사이클 없음	**채택 (O)**	`{A, B}` 연결 (선택 간선 1개, 누적 가중치 = 1)
2	`(B, C) = 2`	사이클 없음	**채택 (O)**	`{A, B, C}` 연결 (선택 간선 2개, 누적 가중치 = 1 + 2 = 3)
3	`(A, C) = 3`	**사이클 발생 (A-B-C-A)**	**기각 (X)**	A와 C가 이미 같은 집합 `{A, B, C}`에 속하므로 제외
4	`(C, D) = 4`	사이클 없음	**채택 (O)**	`{A, B, C, D}` 연결 (선택 간선 3개, 누적 가중치 = 3 + 4 = 7)
5	`(B, D) = 5`	**사이클 발생 (B-C-D-B)**	**기각 (X)**	B와 D가 이미 같은 집합 `{A, B, C, D}`에 속하므로 제외
6	`(D, E) = 6`	사이클 없음	**채택 (O)**	`{A, B, C, D, E}` 연결 (선택 간선 4개 완료 → 종료, **총 가중치 합 = 13**)

- **[3-2] 크루스칼 vs 프림 비교 및 그래프 밀도별 유리성 (2점):**
  1. **동작 방식 및 자료구조 차이:**
     - **크루스칼(Kruskal): 간선(Edge) 중심** 알고리즘으로, 전체 간선을 가중치 오름차순으로 정렬한 뒤 **Union-Find(서로소 집합, Disjoint-Set)** 자료구조를 이용해 사이클을 생성하지 않는 최소 가중치 간선을 차례대로 선택한다(여러 개의 트리 조각들이 합쳐지며 MST 완성).
     - **프림(Prim): 정점(Vertex) 중심** 알고리즘으로, 하나의 시작 정점에서 출발하여 현재까지 구성된 트리에 연결된 인접 정점 중 가장 가중치가 작은 정점을 **우선순위 큐(Min-Heap)**로 선택하며 하나의 트리를 연속적으로 확장한다.
  2. **시간복잡도 및 희소/밀집 그래프 유리성:**
     - 크루스칼의 시간복잡도는 간선 정렬이 지배하므로 $O(|E| \log |E|)$(또는 $O(|E| \log |V|)$)이고, 프림의 시간복잡도는 이진 힙 기준 $O((|V| + |E|) \log |V|)$(인접 행렬 기준 $O(|V|^2)$)이다.
     - 따라서 간선 수가 적은 **희소 그래프(Sparse Graph, $|E| \approx |V|$)에서는 크루스칼 알고리즘**이 유리하고, 간선 수가 정점 수의 제곱에 가까운 **밀집 그래프(Dense Graph, $|E| \approx |V|^2$)에서는 프림 알고리즘**이 간선 전체 정렬 오버헤드가 없어 더 유리하다.

소문항	평가 항목 (채점 키워드)	배점	교차채점 상세 기준 (부분점수 가이드)
3-1	**간선 채택(O) / 기각(X) Trace**	**1점**	채택 간선 4개($(A,B), (B,C), (C,D), (D,E)$)와 사이클로 기각되는 2개($(A,C)=3, (B,D)=5$)를 정확히 구분 시 1점
3-1	**MST 총 가중치 합 산출**	**1점**	$1 + 2 + 4 + 6 = 13$ 정확히 도출 시 1점
3-2	**동작 방식 및 핵심 자료구조 비교**	**1점**	간선 중심(Kruskal) vs 정점 중심(Prim) 구분(0.5점) + 크루스칼 사이클 판별 자료구조 **$Union\text{-}Find(서로소\ 집합)$** 명시(0.5점)
3-2	**시간복잡도 및 희소·밀집 그래프 선택**	**1점**	두 알고리즘 시간복잡도 기재(0.5점) + 희소 그래프 = 크루스칼, 밀집 그래프 = 프림 매칭(0.5점)